

神经网络：从感知机的分类边界到深度学习的特征学习

神经网络与线性回归、对数几率回归、决策树一样，都是用于分类、回归等任务的机器学习方法。理解神经网络，可以从两个相连的问题入手：网络怎样把输入变成输出，又怎样利用训练样本调整这个映射。前一个问题涉及神经元、连接权重和网络层次，后一个问题涉及损失函数、梯度更新和误差逆传播。深度学习延续了这条思路：通过更深的神经网络，让模型在完成任务的同时学习有用的特征。

神经元如何形成分类边界

神经元接收输入，对各个输入按连接权重求和，再结合阈值和激活函数得到输出。《西瓜书》第 5.1 节图 5.1 中的 M-P 神经元模型，以 McCulloch 和 Pitts 两位作者的姓氏首字母命名。这里的“阈值”表示激活所需跨越的界限；“阈”读作 yù，容易与读作 fá 的“阀”混淆。

感知机由两层神经元组成。它把样本特征送入输入层，再由输出神经元作出分类判断。沿着模型、学习策略和学习算法三个环节，可以推得《西瓜书》式 (5.1) 和式 (5.2) 的参数更新关系；这一推导也可参照李航《统计学习方法》[1]。

设输入为特征向量 $x \in \mathbb{R}^n$ ，权重为 $w \in \mathbb{R}^n$ ，阈值为 θ ，激活函数为 f ，感知机的输出写为

$$y = f\left(\sum_{i=1}^n w_i x_i - \theta\right) = f(w^T x - \theta)$$

若取阶跃函数，并用 $\varepsilon(\cdot)$ 表示，输出就只有 0 和 1 两种可能：

$$y = \varepsilon(w^T x - \theta) = \begin{cases} 1, & w^T x - \theta \geq 0; \\ 0, & w^T x - \theta < 0. \end{cases}$$

这一计算具有直接的几何含义。在 n 维空间中，超平面的方程为

$$w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = w^T x + b = 0$$

令 $b = -\theta$ ，便得到感知机的分类边界 $w^T x - \theta = 0$ 。边界将空间分成两部分：满足 $w^T x - \theta \geq 0$ 的样本输出 1，满足 $w^T x - \theta < 0$ 的样本输出 0。等号属于输出 1 的一侧，这一约定也决定了后面怎样判断误分类。

从误分类点到感知机的学习规则

学习目标与线性可分条件

给定训练集

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

其中 $x_i \in \mathbb{R}^n$ ， $y_i \in \{0, 1\}$ ， $i = 1, 2, \dots, N$ ； N 是样本数， n 是每个样本的特征维数。如果存在一个超平面

$$w^T x + b = 0$$

使所有正样本，即 $y_i = 1$ 的样本，都满足 $w^T x_i + b \geq 0$ ，同时所有负样本，即 $y_i = 0$ 的样本，都满足 $w^T x_i + b < 0$ ，就称训练集 T 线性可分；不存在这样的超平面时，称为线性不可分。

以下感知机学习过程以 T 线性可分作为前提，目标是找到一个能把训练集中的正负样本全部分对的分离超平面：

$$w^T x - \theta = 0$$

为了区分模型输出和真实标记，接下来用 $\hat{y}$ 表示预测值，用 y 表示真实标记。设当前参数下的误分类样本集合为 $M \subseteq T$ 。对其中任意一个样本 (x, y) ，错误只有两种情形。当 $w^T x - \theta \geq 0$ 时，模型预测 $\hat{y} = 1$ ，真实标记却是 $y = 0$ ；当 $w^T x - \theta < 0$ 时，模型预测 $\hat{y} = 0$ ，真实标记却是 $y = 1$ 。把两种情形合在一起，可以得到

$$(\hat{y} - y)(w^T x - \theta) \geq 0$$

第一种错误中， $\hat{y} - y = 1$ ，乘上的量非负；第二种错误中， $\hat{y} - y = -1$ ，乘上的量为负，因此乘积仍为正。这给出了构造损失函数的依据：

$$L(w, \theta) = \sum_{x \in M} (\hat{y} - y) (w^T x - \theta)$$

这里的 $\sum_{x \in M}$ 是按误分类样本求和的简写，每个特征向量都与其标记配对。损失非负；如果没有误分类点，求和为 0，损失为 0。误分类点的数量，以及它们偏离分类边界的程度，都会影响损失：误分类点越少、越靠近超平面，损失越小。超平面的距离关系可参见《南瓜书》第 6.1.2 节。

这些性质把“分对所有样本”转成了一个参数优化问题：

$$\min_{w, \theta} L(w, \theta) = \min_{w, \theta} \sum_{x_i \in M} (\hat{y}_i - y_i) (w^T x_i - \theta)$$

求导时需要保留一个条件：先固定当前的误分类集合 M ，在这一条件下，损失关于参数的表达式连续可导。参数更新后， M 也可能变化，因而不能把固定 M 时的求导直接理解成分类边界处处光滑；边界上还可能出现误分类项为 0 的情况。因此，“没有误分类点时损失为 0”并不等于“损失为 0 时一定没有误分类点”。实际迭代仍以误分类集合是否为空来判断训练样本是否全部分对。

把阈值并入权重，再逐点更新

权重和阈值可以用同一套向量记号处理。为输入增加一个固定值为 -1 的“哑节点”，并令其权重为 $w_{n+1} = \theta$ ，就有

$$-\theta = -1 \cdot w_{n+1} = x_{n+1} \cdot w_{n+1}$$

于是，原来的线性计算可以写成

$$\begin{aligned} w^T x_i - \theta &= \sum_{j=1}^n w_j x_j + x_{n+1} \cdot w_{n+1} \\ &= \sum_{j=1}^{n+1} w_j x_j \\ &= w^T x_i \end{aligned}$$

这一步沿用了 x_i 和 w 的记号，但它们的维数已经从 n 变成了 $n+1$ ；最后一维分别为固定输入 -1 和阈值 θ 。等式右端不再单独出现阈值，是因为它已经被合并到内积中。求和中的 x_j 表示当前样本的第 j 个特征分量。

优化问题随之简化为

$$\min_w L(w) = \min_w \sum_{x_i \in M} (\hat{y}_i - y_i) w^T x_i$$

在误分类集合 M 固定的条件下，对权重求导得到

$$\nabla_w L(w) = \sum_{x_i \in M} (\hat{y}_i - y_i) x_i$$

感知机采用随机梯度下降来更新参数。它每次从 M 中随机选择一个误分类点，用这个点对应的梯度作一次更新，而不在每一步都累加全部误分类点的梯度。设学习率为 η ，更新关系为

$$w \leftarrow w + \Delta w$$

$$\Delta w = -\eta(\hat{y}_i - y_i)x_i = \eta(y_i - \hat{y}_i)x_i$$

分量形式就是式 (5.2) 所表达的学习规则。若用 i 标识本次选中的样本，用 j 标识权重分量，则可以避免样本下标与特征下标混淆：

$$\Delta w_j = \eta(y_i - \hat{y}_i)x_{ij}.$$

由于扩充输入的最后一维为 -1，同一个更新关系也包含阈值的调整： $\Delta \theta = -\eta(y_i - \hat{y}_i)$ 。

实际求解时，先随机初始化权重 w_0 ，把训练样本逐一代入模型，确定当前误分类集合 M ；然后随机抽取一个误分类点，计算 Δw ，得到 $w_1 = w_0 + \Delta w$ 。更新之后必须重新确定 M ，再继续选择误分类点、更新权重，直到 M 为空。这里仍依赖前面的线性可分条件，不能把“直到全部分对”不加条件地用于线性不可分的训练集。

感知机的解不唯一。不同的初始权重，以及不同的误分类点选择顺序，都可能使迭代最终得到不同的分离超平面。

隐层怎样完成异或计算

单个感知机用一个超平面完成分类，多层网络则可以先在隐层计算中间结果，再由输出层组合这些结果。《西瓜书》图 5.5 用四个样本 $(0, 0)$ 、 $(0, 1)$ 、 $(1, 0)$ 、 $(1, 1)$ 展示了异或计算。两个隐节点的输出分别为 h_1 和 h_2 ，整个计算关系是

$$(x_1, x_2) \rightarrow h_1 = \varepsilon(x_1 - x_2 - 0.5), h_2 = \varepsilon(x_2 - x_1 - 0.5) \rightarrow y = \varepsilon(h_1 + h_2 - 0.5)$$

h_1 检测 x_1 比 x_2 大的情形， h_2 检测相反的情形，输出节点再把这两个结果合并。以 $(0, 1)$ 为例，先计算

$$\begin{aligned} h_1 &= \varepsilon(0 - 1 - 0.5) = 0, \\ h_2 &= \varepsilon(1 - 0 - 0.5) = 1, \end{aligned}$$

再计算 $y = \varepsilon(0 + 1 - 0.5) = 1$ 。同样代入其余三个样本， $(0, 0)$ 和 $(1, 1)$ 的两个隐节点均输出 0，最终输出也为 0； $(1, 0)$ 的隐节点输出为 $(1, 0)$ ，最终输出为 1。输入不同时输出 1，输入相同时输出 0，正好得到异或关系。

这个例子说明了多层计算如何组合简单的判断，但网络的权重和阈值并不总能手工指定。网络包含隐层之后，学习算法还需要回答：输出端的误差怎样影响前面各层的参数？

误差逆传播：沿计算关系求梯度

误差逆传播算法，即 BP 算法，利用链式法则把输出误差对参数的影响逐层传回。为读懂《西瓜书》第 5.3 节的更新公式，先把符号与网络中的位置对应起来。

用 i 标识输入节点， h 标识隐层节点， j 标识输出节点。 v_{ih} 是输入节点 i 到隐层节点 h 的连接权重， w_{hj} 是隐层节点 h 到输出节点 j 的连接权重； γ_h 和 θ_j 分别是隐层与输出层的阈值。隐层接收到的加权和为 α_h ，隐层输出为 b_h ；输出层接收到的加权和为 β_j 。这些量满足

$$\alpha_h = \sum_i v_{ih}x_i, \quad b_h = f(\alpha_h - \gamma_h), \quad \beta_j = \sum_h w_{hj}b_h, \quad \hat{y}_j^k = f(\beta_j - \theta_j).$$

上标 k 标识训练样本，下标 j 标识该样本的一个输出分量； y_j^k 是真实值， $\hat{y}_j^k$ 是预测值。若输出节点共有 l 个，样本 k 的平方误差为

$$E_k = \frac{1}{2} \sum_{j=1}^l (\hat{y}_j^k - y_j^k)^2.$$

以下推导使用满足 $f'(z) = f(z)[1 - f(z)]$ 的 Sigmoid 激活函数。这一导数关系是后面得到 $\hat{y}_j^k(1 - \hat{y}_j^k)$ 和 $b_h(1 - b_h)$ 的条件，不能直接套用到前面感知机的阶跃函数上。

输出层阈值：式 (5.10) 与式 (5.12)

梯度下降要求沿误差减小的方向调整阈值：

$$\Delta \theta_j = -\eta \frac{\partial E_k}{\partial \theta_j}$$

阈值 θ_j 通过 $f(\beta_j - \theta_j)$ 影响第 j 个输出，而该输出又影响误差 E_k 。沿这条关系使用链式法则，有

$$\begin{aligned}
\frac{\partial E_k}{\partial \theta_j} &= \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \theta_j} \\
&= \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial [f(\beta_j - \theta_j)]}{\partial \theta_j} \\
&= \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot f'(\beta_j - \theta_j) \times (-1) \\
&= \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot f(\beta_j - \theta_j) \times [1 - f(\beta_j - \theta_j)] \times (-1) \\
&= \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \hat{y}_j^k (1 - \hat{y}_j^k) \times (-1) \\
&= \frac{\partial \left[\frac{1}{2} \sum_{j=1}^l (\hat{y}_j^k - y_j^k)^2 \right]}{\partial \hat{y}_j^k} \cdot \hat{y}_j^k (1 - \hat{y}_j^k) \times (-1) \\
&= \frac{1}{2} \times 2(\hat{y}_j^k - y_j^k) \times 1 \cdot \hat{y}_j^k (1 - \hat{y}_j^k) \times (-1) \\
&= (y_j^k - \hat{y}_j^k) \hat{y}_j^k (1 - \hat{y}_j^k) \\
&= g_j
\end{aligned}$$

推导中的负号来自 $\partial(\beta_j - \theta_j)/\partial \theta_j = -1$ 。对平方误差求导则得到 $\hat{y}_j^k - y_j^k$ ，二者结合后，最终得到

$$g_j = (y_j^k - \hat{y}_j^k) \hat{y}_j^k (1 - \hat{y}_j^k).$$

这一中间量对应式 (5.10)，其推导已经包含在式 (5.12) 的推导中。将 $\partial E_k / \partial \theta_j = g_j$ 代回梯度下降式，即得输出层阈值的更新：

$$\Delta \theta_j = -\eta \frac{\partial E_k}{\partial \theta_j} = -\eta g_j$$

输入到隐层的权重：式 (5.13) 与式 (5.15)

调整 v_{ih} 时，影响要先经过隐层加权和 α_h 和隐层输出 b_h ，再传到输出层加权和 β_j 、预测值 $\hat{y}_j^k$ ，最后改变误差。由于同一个隐节点会影响多个输出节点，需要把通向全部 l 个输出的贡献相加。

梯度下降的起点仍是

$$\Delta v_{ih} = -\eta \frac{\partial E_k}{\partial v_{ih}}$$

完整展开链式求导，得到

$$\begin{aligned}
\frac{\partial E_k}{\partial v_{ih}} &= \sum_{j=1}^l \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial b_h} \cdot \frac{\partial b_h}{\partial \alpha_h} \cdot \frac{\partial \alpha_h}{\partial v_{ih}} \\
&= \sum_{j=1}^l \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial b_h} \cdot \frac{\partial b_h}{\partial \alpha_h} \cdot x_i \\
&= \sum_{j=1}^l \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial b_h} \cdot f'(\alpha_h - \gamma_h) \cdot x_i \\
&= \sum_{j=1}^l \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot w_{hj} \cdot f'(\alpha_h - \gamma_h) \cdot x_i \\
&= \sum_{j=1}^l (-g_j) \cdot w_{hj} \cdot f'(\alpha_h - \gamma_h) \cdot x_i \\
&= -f'(\alpha_h - \gamma_h) \cdot \sum_{j=1}^l g_j \cdot w_{hj} \cdot x_i \\
&= -b_h(1 - b_h) \cdot \sum_{j=1}^l g_j \cdot w_{hj} \cdot x_i \\
&= -e_h \cdot x_i
\end{aligned}$$

这里先由 $\partial \alpha_h / \partial v_{ih} = x_i$ 引入输入分量，再由 $\partial \beta_j / \partial b_h = w_{hj}$ 引入隐层到输出层的权重。输出端的两项导数乘积等于 $-g_j$ ，而隐层激活函数的导数为 $b_h(1 - b_h)$ 。因此，可把隐层节点对应的误差信号定义为

$$e_h = b_h(1 - b_h) \sum_{j=1}^l w_{hj} g_j.$$

这就是式 (5.15) 中的量，也说明为什么它的推导可以与式 (5.13) 一起完成： e_h 是把各输出节点的误差信号按连接权重汇总，再乘以隐节点自身的激活导数。由 $\partial E_k / \partial v_{ih} = -e_h x_i$ 可得

$$\Delta v_{ih} = -\eta \frac{\partial E_k}{\partial v_{ih}} = \eta e_h x_i$$

隐层阈值：式 (5.14)

隐层阈值 γ_h 也会经由 b_h 影响各个输出节点，因此仍须累加所有输出路径上的贡献。它的更新起点为

$$\Delta \gamma_h = -\eta \frac{\partial E_k}{\partial \gamma_h}$$

与权重求导相比，关键变化在于 $\partial(\alpha_h - \gamma_h) / \partial \gamma_h = -1$ 。逐步计算为

$$\begin{aligned}
\frac{\partial E_k}{\partial \gamma_h} &= \sum_{j=1}^l \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial b_h} \cdot \frac{\partial b_h}{\partial \gamma_h} \\
&= \sum_{j=1}^l \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \frac{\partial \beta_j}{\partial b_h} \cdot f'(\alpha_h - \gamma_h) \cdot (-1) \\
&= -\sum_{j=1}^l \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot w_{hj} \cdot f'(\alpha_h - \gamma_h) \\
&= -\sum_{j=1}^l \frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot w_{hj} \cdot b_h(1 - b_h) \\
&= \sum_{j=1}^l g_j \cdot w_{hj} \cdot b_h(1 - b_h) \\
&= e_h
\end{aligned}$$

于是隐层阈值的更新为

$$\Delta \gamma_h = -\eta \frac{\partial E_k}{\partial \gamma_h} = -\eta e_h$$

这三组推导沿用同一个原则：先找到参数到误差的所有影响路径，再按链式法则相乘、求和，最后乘上梯度下降的 $-\eta$ 。阈值在神经元输入中以减号出现，因此不能直接照搬连接权重更新式的符号。

梯度下降能否找到全局最小

推导出梯度，并不意味着已经解决了优化的全部困难。《西瓜书》第 5.4 节图 5.10 将局部极小与全局最小放在一起比较：局部极小只要求在某个邻域内没有更小的函数值，全局最小则要求在整个考察范围内都没有更小的函数值。一个位置在附近已经无法继续下降，并不足以说明它优于所有其他位置。

这一区别也限制了对 BP 学习结果的理解：梯度提供当前参数附近的变化方向，不能仅凭局部下降过程就断言得到全局最优解。模拟退火、遗传算法以及其他启发式方法涉及更广泛的优化思路，若要理解其具体机制和适用条件，还需查阅专业资料系统学习。

其他网络的两端理解方式

《西瓜书》第 5.5 节介绍了其他神经网络。与第 5.6 节涉及的卷积神经网络、循环神经网络等相比，这一节中的网络如今已不太常见。不过，它们有助于理解神经网络与其他模型之间的联系：RBF 网络可以从特征变换和线性回归来理解，Boltzmann 机则可以从无向图和能量来理解。

RBF 网络：确定中心之后做线性回归

式 (5.18) 中，RBF 网络对样本 x 的输出，是 q 个 $\rho(x, c_i)$ 的线性组合。 c_i 表示第 i 个神经元中心， w_i 是相应的线性加权系数，因此输出可写成

$$f(x) = \sum_{i=1}^q w_i \rho(x, c_i).$$

也可以把这些响应看成新的特征。按照式 (5.19) 对 d 维输入 x 作特征转换，得到 q 维向量

$$\tilde{x} = (\rho(x, c_1); \rho(x, c_2); \dots; \rho(x, c_q)).$$

这样，求解式 (5.18) 中的 w_i 就可以与第 3.2 节的线性回归对应起来：

$$f(\tilde{x}) = w^T \tilde{x} + b.$$

二者仅差一个偏置项 b ，而式 (5.18) 也可以补加这一项。因此，RBF 网络在确定 q 个神经元中心 c_i 之后，接下来的权重求解就是线性回归。这里的“确定中心之后”是这一对应关系的前提。

Boltzmann 机：把边与节点的能量相加

Boltzmann 机可以理解为引入隐变量的无向图模型。式 (5.20) 的能量由两部分组成：

$$E_{\text{graph}} = E_{\text{edges}} + E_{\text{nodes}}$$

其中， E_{graph} 是整张图的能量， E_{edges} 是所有边的能量之和， E_{nodes} 是所有节点的能量之和。设节点状态为 s_i ，连接权重为 w_{ij} ，阈值为 θ_i ，单条边的能量是 $E_{\text{edge}_{ij}} = -w_{ij}s_i s_j$ ，单个节点的能量是 $E_{\text{node}_i} = -\theta_i s_i$ 。

如果图中共有 n 个节点，边能量求和时取 $i < j$ ，使每条无向边只计一次：

$$E_{\text{edges}} = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E_{\text{edge}_{ij}} = - \sum_{i=1}^{n-1} \sum_{j=i+1}^n w_{ij} s_i s_j$$

节点能量则逐个累加：

$$E_{\text{nodes}} = \sum_{i=1}^n E_{\text{node}_i} = - \sum_{i=1}^n \theta_i s_i$$

两部分相加，就得到状态向量 s 对应的 Boltzmann 机能量：

$$E_{\text{graph}} = E_{\text{edges}} + E_{\text{nodes}} = - \sum_{i=1}^{n-1} \sum_{j=i+1}^n w_{ij} s_i s_j - \sum_{i=1}^n \theta_i s_i$$

受限 Boltzmann 机 (Restricted Boltzmann Machine, RBM) 进一步限制了连接结构：只保留显层与隐层之间的连接。设显层状态向量为 $v = (v_1; v_2; \dots; v_d)$ ，隐层状态向量为 $h = (h_1; h_2; \dots; h_q)$ 。显层内没有相互连接，因此在给定整个隐层状态 h 的条件下，显层变量 $v_1, v_2, \dots, v_d$ 相互独立，这解释了式 (5.22)。

同理，在给定显层状态 v 的条件下，隐层变量 $h_1, h_2, \dots, h_q$ 相互独立，这解释了式 (5.23)。两处独立性都带有“给定另一层状态”的条件，不能省去这一条件而理解为无条件独立。

从多层网络到深度学习

网络深度与历史转折

深度学习以很深的神经网络为基础，因此属于机器学习的子集。网络层次的加深提供了获得更好效果的理论方向，而计算能力的提升使深层网络的训练和应用更为可行。讨论深度时仍需联系前面的学习与优化问题：层数描述网络的结构，实际效果还要通过参数学习来实现。

经典神经网络和 BP 算法并非在深度学习受到广泛关注之后才出现。LeCun 等人的卷积神经网络工作发表于 1989 年 [2]，Rumelhart、Hinton 和 Williams 关于 BP 的工作发表于 1986 年 [3]。在当时的计算能力下，支持向量机等非神经网络算法的效果优于神经网络，神经网络的发展因而进入瓶颈期。

随着计算能力不断提高，转折出现在 2012 年：Hinton 和他的学生提出 AlexNet，在 ImageNet 评测中夺冠，成绩明显优于第二名，由此引起学术界与工业界的广泛关注。2015 年，LeCun、Bengio 和 Hinton 三位被称为“深度学习之父”的学者正式提出深度学习的概念，此后深度学习成为机器学习的主流研究方向。

《西瓜书》第 5.6 节着重从宏观角度解释深度学习，没有逐一展开经典网络的结构。卷积神经网络、循环神经网络等具体模型，需要结合其他相关书籍系统学习。理解这一节的重点，可以先放在“特征由谁来设计”上。

从人工特征工程到网络特征学习

以西瓜分类为例，使用非深度学习算法时，通常先由人设计特征，例如根蒂、色泽，再把它们表示为数学向量。这些工作统称为特征工程。完成特征工程后，才把特征交给算法进行分类；分类效果很大程度上取决于人工选择和构造的特征是否合适。

使用深度学习时，可以把西瓜图片表示为数学向量输入网络，并把输出层设置为需要的分类结果。例如，二分类的输出通常采用对数几率回归。此前由人完成的特征工程，转为由神经网络完成的特征学习：通过输出层对分类结果的约束，网络从图片中自动提取有助于西瓜分类的特征。

因此，对数几率回归与卷积神经网络的比较，还需要说明送入分类器的特征从何而来。采用人工特征时，过程是

人工特征工程 $\rightarrow$ 对数几率回归分类.

采用卷积神经网络时，过程则是

卷积神经网络特征学习 $\rightarrow$ 对数几率回归分类.

两条路径都可以在最后使用对数几率回归，区别在于前面的特征如何形成。深度网络把特征的形成纳入了学习过程，前面各层的参数再通过输出误差的逆向传播得到调整。这样，神经元计算、多层表示、损失函数和 BP 更新便共同构成了从输入图片到分类结果的学习过程。

参考文献

- [1] 李航. 统计学习方法. 清华大学出版社, 2012.
- [2] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4):541–551, 1989.
- [3] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.